

- Why This Project Matters — And Why It's Great for You
 - The Industry Opportunity
 - What You'll Actually Do
 - Why This Beats a Software Internship
 - Career Paths This Enables
- Extending CPUs Pipelines with Sparse Linear Algebra Support and Vector Engines
- Project Summary
- Background & Project Context
- Research Objectives
 - ISA & Programming Model Extensions
- Vector Engine & Pipeline Integration
- Compiler & Runtime Co-Design
- Evaluation & Analysis
- Student Responsibilities
- Supervision & Training Environment
- Milestones
- Anticipated Outcomes
- Required Qualifications
- Preferred / Asset Qualifications

Why This Project Matters — And Why It's Great for You

The Industry Opportunity

The computing industry is undergoing a fundamental shift. As Moore's Law slows, companies like **Apple, Google, NVIDIA, AMD, Intel, and Meta** are racing to build specialized processors for AI and sparse workloads. RISC-V — the open-source ISA — is at the center of this transformation, with billions of chips shipped and growing adoption in everything from embedded devices to datacenter accelerators.

Sparse computation is everywhere:

- Graph neural networks (social networks, drug discovery)
- Recommendation systems (Netflix, Amazon, TikTok)
- Scientific simulations (climate modeling, physics)
- Large language models (transformer attention is fundamentally sparse)

Yet today's processors waste 50–90% of their compute on zeros. This project tackles that directly.

What You'll Actually Do

Unlike many research projects where undergrads spend months just setting up tools, **you'll be doing real architecture research from week one**. Our group has already built:

- A working out-of-order RISC-V simulator
- A compiler pipeline (MLIR/xDSL) for lowering sparse code
- Benchmarking infrastructure

Your job is to **design the future** — propose new instructions, model their execution, and measure whether they work.

Why This Beats a Software Internship

Typical SWE Internship	This Research Project
Fix bugs in existing codebase	Design new ISA extensions
Learn one company's stack	Learn skills used across the industry
Limited ownership	Your ideas become the project
Resume line	Publication potential + strong grad school letter

Career Paths This Enables

This USRA is particularly well-suited for students interested in:

- **Graduate research** (PhD/MSc in architecture, compilers, or ML systems)
- **CPU/GPU design roles** (Apple, AMD, NVIDIA, Intel, Qualcomm)
- **ML accelerator teams** (Google TPU, AWS Trainium, Meta MTIA)
- **Systems research labs** (MSR, Google DeepMind, NVIDIA Research)

Extending CPUs Pipelines with Sparse Linear Algebra Support and Vector Engines

Supervisor: Dr. Arrvindh Shriraman **Department:** School of Computing Science **Institution:** Simon Fraser University **Position Type:** NSERC Undergraduate Student Research Award (USRA) **Duration:** 16 Weeks (Full-Time) **Location:** SFU Burnaby Campus **Start Date:** Summer 2026 (Tentative)

Project Summary

Modern workloads in machine learning, graph analytics, and scientific computing are increasingly **sparse** and irregular. Traditional dense linear algebra execution wastes performance and energy when operating on mostly-zero data. Meanwhile, RISC-V offers an open platform to design and evaluate new instruction set extensions and execution pipelines targeted at these next-generation workloads.

This project investigates how RISC-V pipelines can be extended with:

- **Sparse linear algebra acceleration**
- **Vector execution engines**
- **New ISA semantics that treat sparsity as a first-class architectural concept**

The goal is to design new ISA primitives, integrate them into an out-of-order RISC-V core with a vector engine, build compiler support, and evaluate performance across sparse workloads.

Background & Project Context

This project builds on strong prior work in our group. In earlier research and directed projects (including CMPT 415), **Shawn Lu** (an undergrad student from CMPT 295) developed a robust architectural research platform that the USRA student will directly benefit from:

- an **out-of-order RISC-V timing + functional simulator**
- support for **custom instruction extensions** and experimental pipeline structures
- a **compiler lowering pipeline (MLIR/xDSL)** that maps high-level linear algebra to new ISA constructs
- benchmarking harnesses and evaluation tooling

This infrastructure means the project does **not** start from scratch. Instead, the USRA student can focus on **research innovation** — designing ISA extensions, exploring microarchitectural trade-offs, and studying how architectures should evolve for sparse computation — rather than spending months just building tools.

Research Objectives

The project will explore a rich design space spanning ISA, microarchitecture, and compiler co-design.

ISA & Programming Model Extensions

Student will design and evaluate some combination of:

- **Format-Aware Sparse ISA Support**
 - CSR / CSC / Block sparse / structured N:M sparsity instructions
 - Hybrid dense–sparse execution switching
- **First-Class Sparsity Metadata**
 - architectural sparsity descriptors
 - possible metadata caching
- **Advanced Mask & Predicate Semantics**
 - mask registers
 - compressed mask operations
 - execution compaction
- **Irregular SIMD / Lane Grouping**
 - live-lane packing + restore
 - lane remapping to improve locality
- **Sparse Micro-Kernel Invocation**
 - tasklet-style execution via ISA queues
- **Sparse Memory Semantics**
 - gather/scatter enhancements
 - ISA-visible prefetch and locality hints
- **Format Negotiation & Conversion Assist**
- **Correctness & Stability Modes**

- compensated accumulation
- safe sparse indexing

- **Tensor / Vector Hybrid Execution Bridges**

Specific directions will depend on feasibility and student interest.

Vector Engine & Pipeline Integration

- integrate chosen ISA concepts into OoO RISC-V pipeline
 - model vector lanes, VRF pressure, latency + bandwidth
 - explore tightly-coupled vs coprocessor-style integration
-

Compiler & Runtime Co-Design

- extend existing MLIR/xDSL pipeline
 - map sparse kernels (SpMV, SpMM, graph ops) to new ISA
 - explore auto-format selection and tiling strategies
-

Evaluation & Analysis

- benchmark across sparse scientific + ML + graph workloads
 - compare:
 - baseline RISC-V
 - RISC-V Vector extension
 - proposed sparse/vector extensions
 - analyze:
 - speedup
 - utilization
 - sensitivity to sparsity patterns
-

Student Responsibilities

The USRA student will:

- design new sparse/vector ISA extensions
 - extend simulator + add execution logic
 - build compiler lowering support
 - implement and run benchmarks
 - analyze pipeline behavior and bottlenecks
 - produce a final technical report and presentation
-

Supervision & Training Environment

The student will join an active research environment in computer architecture, accelerators, and ML systems. They will receive:

- weekly one-on-one supervision
- structured onboarding to Shawn Lu’s simulator + compiler tools
- design discussions on sparse architectures and vector engines
- guidance on experiment design and research writing

Because extensive infrastructure already exists, students can meaningfully contribute to architecture ideas within the **first few weeks** of the project.

Milestones

Milestone	Focus
M1	Baseline simulator + benchmarks running
M2	Identify target sparse kernels & formats
M3	Design initial sparse ISA extensions
M4	Implement ISA + execution support
M5	Add compiler lowering
M6	Core evaluation
M7	Optional: mask compaction, metadata caching, or structured sparsity
M8	Final research analysis & report

Anticipated Outcomes

Student gains experience in:

- ISA + microarchitecture design
- vector + sparse compute architecture
- compiler integration
- benchmarking + architectural analysis

Deliverables:

- extended simulator
 - compiler passes
 - benchmark suite
 - evaluation report
-

Required Qualifications

- senior undergraduate in CS, Engineering, or related field
- strong programming in **C/C++** and **Python**

- coursework in computer architecture (e.g., CMPT 295)
 - comfort with Linux environments
-

Preferred / Asset Qualifications

(Not required, but beneficial)

- RISC-V / assembly experience
 - compiler frameworks (LLVM / MLIR / xDSL)
 - simulation / architecture modeling
 - familiarity with sparse linear algebra or vector architectures
-